AfriSommelier — Firebase on Vercel Setup Guide

Repository: Neurogrowth-Labs/AfriSommelier-

What's in the repo

AfriSommelier is a React 19 + Vite + TypeScript app (originally scaffolded in Google AI Studio).

Area	Detail
Framework	Vite (vite.config.ts), React, Tailwind v4
AI	@google/genai (Gemini) and OpenRouter (src/services/openRouterService.ts)
Backend	Firebase — Auth (Google sign-in) + Firestore
Firebase init	src/firebase.ts imports firebase-applet-config.json directly
Firestore schema	Defined in firebase-blueprint.json (users, profile, wishlist, cellar, consumption, events)
Security rules	firestore.rules (per-user RBAC, admin email check)
Build	npm run build → outputs to dist/
Env vars used	GEMINI_API_KEY, OPENROUTER_API_KEY, APP_URL

Note: The file `firebase-applet-config.json` (containing apiKey, projectId, etc.) is currently committed to the repo. Firebase web API keys are not strictly secret (Firestore rules are what protect data), but it is cleaner to move them to env vars before going to production.

Setting up Firebase on Vercel

There are two halves to "Firebase on Vercel": (1) making the Vercel-hosted frontend talk to Firebase, and (2) configuring Firebase itself to trust your Vercel domain.

1. Prepare a Firebase project

If you want to keep using the existing AI Studio project (gen-lang-client-0322601470), skip to step 2. Otherwise:

- Go to https://console.firebase.google.com → Add project.
- In Project settings → General → Your apps, register a Web app and copy the config (apiKey, authDomain, projectId, storageBucket, messagingSenderId, appId, measurementId).
- Enable Authentication → Sign-in method → Google.
- Create a Firestore database (Native mode). The repo currently uses firestoreDatabaseId "ai-studio-..."; either create that named database, or use (default) and update the code accordingly.
- Deploy the rules from firestore.rules:

```
npm i -g firebase-tools
firebase login
firebase init firestore   # pick your project, point rules to firestore.rules
firebase deploy --only firestore:rules
```

■ **Warning:** The admin check in firestore.rules (line 69) hardcodes `lusimadio12@gmail.com` — change it to your admin email before deploying.

2. Import the Vercel project

- Push the repo to GitHub (already done).
- Open https://vercel.com/new → Import Git Repository → pick AfriSommelier-.
- Vercel auto-detects Vite. Confirm: Framework preset = Vite, Build command = npm run build, Output directory = dist, Install command = npm install.

3. Add environment variables in Vercel

In the Vercel project: Settings → Environment Variables (set them for Production, Preview, and Development):

Name	Value	Why
GEMINI_API_KEY	your Gemini key	Read by vite.config.ts and exposed via process.env.GEMINI_API_KE
OPENROUTER_API_KEY	your OpenRouter key	Used in src/services/openRouterService.ts
APP_URL	https://<your-project>.vercel.app	Sent as HTTP-Referer to OpenRouter

Recommended: Switch from the committed JSON to env-vars by also adding:

```
VITE_FIREBASE_API_KEY=...
VITE_FIREBASE_AUTH_DOMAIN=<project>.firebaseapp.com
VITE_FIREBASE_PROJECT_ID=...
VITE_FIREBASE_STORAGE_BUCKET=<project>.firebasestorage.app
VITE_FIREBASE_MESSAGING_SENDER_ID=...
VITE_FIREBASE_APP_ID=...
VITE_FIREBASE_MEASUREMENT_ID=...
VITE_FIREBASE_DATABASE_ID=(default)   # or your custom DB id
```

Then change src/firebase.ts to read from import.meta.env.VITE_FIREBASE_* instead of importing firebase-applet-config.json, and remove that JSON file from the repo. The VITE_ prefix is required so Vite exposes them to the browser bundle.

4. Authorize the Vercel domain in Firebase

This step is commonly forgotten and breaks Google sign-in in production:

- Firebase Console → Authentication → Settings → Authorized domains → Add domain:
- – your-project.vercel.app
- – Each Preview URL pattern you care about (e.g. your-project-git-main-username.vercel.app)
- – Your custom domain if you attach one
- Google Cloud Console → APIs & Services → Credentials → OAuth 2.0 Client (Web client) backing Firebase Auth → add the same URLs to Authorized JavaScript origins. The redirect URI https://.firebaseapp.com/__/auth/handler is already configured by Firebase.

5. Restrict the Firebase API key (optional but recommended)

In Google Cloud Console → APIs & Services → Credentials, edit the Browser API key:

- Application restrictions → HTTP referrers → add https://*.vercel.app/* and your custom domain.
- API restrictions → limit to: Identity Toolkit API, Token Service API, Cloud Firestore API, Firebase Installations API.

6. Deploy

Push to main (or click Deploy in Vercel). After the build completes:

- Open the Vercel URL.
- Try Google sign-in — if you see auth/unauthorized-domain, revisit step 4.
- Open DevTools → Network and confirm Firestore requests succeed; if you get permission-denied, revisit the rules (step 1) or your firestoreDatabaseId.

7. Things that will not work on Vercel out of the box

- express is in package.json but no server is wired up; Vercel's static Vite output is fine for this app. If you later add an Express API, move it to api/*.ts Vercel Functions or use a separate service.
- The APP_URL env var must be updated for each environment (Production vs Preview) if OpenRouter referer accuracy matters.
- The committed firebase-applet-config.json will keep working on Vercel, but I recommend removing it once you switch to VITE_FIREBASE_* env vars so config is environment-specific.

TL;DR checklist

- ■ Firebase project created, Google auth enabled, Firestore created
- ■ firestore.rules deployed (admin email updated)

■ Repo imported into Vercel (Vite preset, default build)

■ Env vars set in Vercel: GEMINI_API_KEY, OPENROUTER_API_KEY, APP_URL, and (recommended) VITE_FIREBASE_*

■ src/firebase.ts updated to read env vars (if you went the recommended route)

■ Vercel domain(s) added to Firebase Authorized domains

■ Firebase API key restricted to your domains

■ Deploy, test Google sign-in + a Firestore read/write